Journal of Heuristics (2023) 29:177–206
https://doi.org/10.1007/s10732-023-09509-8

Minimizing total completion time in large-sized pharmaceutical quality control scheduling

Miguel S. E. Martins[1] · Joaquim L. Viegas[1] · Tiago Coito[1] · Bernardo Firme[1] · Andrea Costigliola[2] · João Figueiredo[1] · Susana M. Vieira[1] · João M. C. Sousa[1]

Received: 31 May 2021 / Revised: 7 December 2022 / Accepted: 21 January 2023 /
Published online: 14 February 2023
© The Author(s) 2023

Abstract
This paper proposes an algorithm for the effective scheduling of analytical chemistry tests in the context of quality control for pharmaceutical manufacturing. The problem is formulated as an extension of a dual resource constrained flexible job shop scheduling problem for the allocation of both machines and analysts resources for analytical laboratory work of real dimensions. The formulation is novel and custom made to fit real quality control laboratory. The novelty comes from allowing multiple analyst interventions for each machine allocation while minimising the total completion time, formulated as a mixed integer linear programming model. A three-level dynamic heuristic is proposed to solve the problem efficiently for instances representative of real world schedules. The CPLEX solver and a Tabu Search algorithm are used for comparison. Results show that the heuristic is competitive with the other strategies for medium-sized instances while outperforming them for large-sized instances. The dynamic heuristic runs in a very short amount of time, making it suitable for real world environments. This work is valuable for the development of laboratory management solutions for quality control as it presents a way to provide automatic scheduling of resources.

Keywords Scheduling · Heuristics · Dual resource constrained scheduling · Quality control laboratory

✉ Miguel S. E. Martins
miguelsemartins@tecnico.ulisboa.pt

[1] IDMEC, Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal

[2] Hovione Farmaciência, S.A., Lisbon, Portugal

1 Introduction

In manufacturing, quality control (QC) is responsible for ensuring that goods comply with predefined standards. QC encompasses a set of infrastructures, guidelines and practices concerned with achieving the desired quality in produced goods (Montgomery 2005). More specifically, it is responsible for monitoring manufacturing processes through the assessment of samples taken at different stages of the production process. Samples can be taken from raw materials that require transformation, from parts produced or obtained from suppliers and finished products. Tests are performed on samples to assess different quality metrics that are compared to pre-determined standards. In this way, if QC works effectively, deviations in manufacturing can be promptly detected, minimising the negative impacts downstream. The role of QC is fundamental in many industries (Geigert 2002).

The pharmaceutical industry, more specifically the manufacturing of drugs and medications, consists of complex processes that require a high precision in control. Quality control is critical in drug manufacturing because the processes typically deal with large batch sizes, make use of dangerous materials and processing steps can be long. The lack of effective QC can have impacts ranging from significant delays in production to dangers to public health. Pharmaceutical companies operate in one of the most regulated markets, and compliance with Good Manufacturing Practices is required to provide pharmaceutical goods to the market, ensuring the safety and quality requirements (Costigliola et al. 2017; Lopes et al. 2020).

In particular for the pharmaceutical industry, QC labs are in charge of running analytical chemistry tests on samples coming from different involved parties and stages of the manufacturing process. The samples are released by the laboratories if results comply with the required specifications, letting manufacturing continue or releasing the product to go to market. The quality control test are often destructive or pollute the original sample. Thus, what is called a sample is often a bundle of individual sample vials taken at the same time, allowing multiple tests of uncontaminated product samples.

When dealing with generic or contract manufacturing of pharmaceuticals, the QC lab environment is specially complex as the mix of products is greater and visibility of incoming samples is reduced. A manufacturing batch is a large set of samples in a laboratorial context. Each batch results in samples of raw materials, samples of in process testing, finished product and stability. Each sample represents several tests, each requiring specific instruments and skills. Weekly lab activities can amount to the tens of thousands, which combined with the specific requirements on instruments and skills make the management of QC labs complex. Currently, laboratories are mainly managed through manual scheduling techniques and time consuming communication methods (Maslaton 2012; Costigliola et al. 2017).

With the advent of the Industry 4.0 concept (Lv and Lin 2017; Zheng and Wu 2017), many opportunities for improvement are available in laboratories. Labs can leverage Internet of Things, location detection, human machine interfaces and high performance databases to have a real-time overview of all activities (Coito et al. 2020). Using this information, one of the most significant opportunities for improvement in the management of labs is associated with the use of automatic scheduling algorithms

(Maslaton 2012). Complex scheduling problems have been studied extensively in the literature since the sixties (Pinedo 2016), but research focused on quality control laboratories scheduling is scarce (Ruiz-Torres et al. 2012).

The main objective of this paper is to propose a solution to automatically schedule real-sized QC labs. This is achieved by dealing with two sub-objectives: 1) formulation of the problem; and 2) development of an algorithm to generate good quality solutions in a suitable amount of time for real-world applications. The mathematical formulation and proposed heuristic are novel and improve upon current literature to solve the dual resource constrained flexible job shop scheduling problem.

The motivation is the optimization of a real-world scheduling problem of laboratory analysis at Hovione (Costigliola et al. 2017; Lopes et al. 2020?; ?). The emphasis on large-sized instances is motivated by the complexity inherent of real world pharmaceutical quality control laboratories.

The problem is formulated as a Mixed Integer Linear Programming (MILP) model in which a set of jobs needs to be assigned to resources of two types: machines and workers. The formulation is also flexible regarding worker tasks. Each operation of a job can include any number of worker tasks at predetermined time points. Additionally, workers may switch between tasks of different operations if possible. This formulation extends existing literature formulations by changing the objective function and presenting a simpler notation, resulting in fewer variables and constraints for the same problems.

Exact methods become unsuitable to solve realistic problems as their size and complexity increases (Zhang et al. 2017; Flores-Luyo et al. 2020). Thus, a solution method is proposed consisting of a three-level dynamic heuristic due to how quickly and reliably heuristics can produce good solutions for large instances. This heuristic builds upon common scheduling heuristics to deal with the DRC FJSP. Then, a Tabu Search is implemented for comparison with the previous methods.

The remainder is structured as follows: Section 2 reviews the main literature on QC lab scheduling and DRC FJSP; Section 3 presents the detailed description of the problem and mathematical model; Section 4 details the proposed three-level dynamic heuristic; Section 5 details the implemented Tabu Search; Section 6 explains the computational experiments and discusses the results; and Section 7 presents the main conclusions of this paper.

2 Literature review

2.1 Quality control laboratory scheduling

Regarding the topic of QC lab scheduling, current literature points at the importance of developing automated tools to improve efficiency and reduce managerial load. Schäfer (2004) explains the complexity in QC lab scheduling, detailing the different concepts that require consideration if dynamic scheduling is to be implemented. It identifies all the components interacting on the lab workbench, the flows of information between stakeholders, activity workflows and the different constraints related to both sample characteristics, worker competences and instrument management.

Maslaton (2012) presents the strategic view and the role of scheduling in QC labs. It points out the challenges that come from complex industries such as contract and generic pharmaceutical manufacturing. Additionally, the author presents practical considerations for the development of a lab scheduling tool, including integration with common systems in the labs such as Lab Information Management Systems (LIMS). However, the author does not propose a method to automatically create schedules based on the characteristics of the entities involved in the lab. The author is also associated with the commercial QC lab scheduling tool SmartQC (CResults 2019).

Costigliola et al. (2017) uses a simulation model to investigate the effect of different scheduling policies in QC lab efficiency. The results show that the use of batching for tests with similar characteristics can result in significant improvements in the average processing time.

Similarly to the approach proposed in this paper, Ruiz-Torres et al. (2012) also deals with QC lab scheduling as a typical Operations Research (OR) scheduling problem. The authors also consider that specific skills are required for each test and that each test is divided in multiple tasks but, contrary to our approach, only consider the assignment of workers. Considering the assignment of machines, which results in the same type of constraints as workers, results in a much greater complexity. The authors propose a dynamic heuristic based on list scheduling rules, which consists in ordering the list of tasks in a certain manner and iteratively constructing the schedule based on a set of rules. Ordering of tasks regarding their due date, number of remaining tests of the associated job, number of available technicians and arrival time are investigated. In Ruiz-Torres et al. (2017) the same author deals with the problem with the objective of maximising the overall technician preference index.

Quality control laboratories in chemical manufacturing often suffer from long processing times. A considerable number of samples stay in the laboratory more than 24 hours (Costigliola et al. 2017), often leaving unprocessed samples waiting for analysts. Since jobs are very lengthy, an efficient machine allocation is extremely important to avoid long queues. Analysts are crucial for setting up and confirm the results of the machines (Lopes et al. 2018), thus motivating the use of dual-resource constrained formulation. Also, scheduling laboratory tasks by allocating analysts and machines mimics real world manual scheduling procedures on QC laboratories.

2.2 Dual resource constrained flexible job shop scheduling

Classical scheduling literature deals with problems in which there is a finite number of jobs and machines with the objective of assigning them to each-other while minimising or maximising some criteria (Pinedo 2016). More specifically, the job shop scheduling problem (JSP) considers an environment with m machines where each job follows a predetermined route in the system. The JSP is known to be strongly NP-Hard (Garey et al. 1976). As a generalization of the JSP, the flexible job shop problem (FJSP) considers that instead of having m different machines, jobs can be processed by any machine from a given set a set of resources (Pinedo 2016). In QC scheduling, each test needs to be done in a specific type of machine and for each type multiple machines can exist.

The FJSP has been extensively investigated in the literature (Chaudhry and Khan 2016). Good results have been achieved using different techniques, including the use of MILP (Roshanaei et al. 2013), metaheuristics such as Genetic Algorithms (GA) (Ho et al. 2007) and Tabu Search (TS) (Saidi-Mehrabad and Fattahi 2007), and many others. Additionally, more complex extensions of the problem have been dealt with, such as considering sequence-dependent setup times (Shen et al. 2018), blocking constraints (Mati et al. 2011) and multi-criteria objectives (Guimarães and Fernandes 2007; Ahmadi et al. 2016).

A further generalisation of JSP is to consider the simultaneous scheduling of more than one resource. In these cases a primary resource, usually a machine, might need to be allocated simultaneously with some other auxiliary resources (Bitar et al. 2016; Burdett et al. 2019, 2020), making the problem extremely complex.

The dual resource constrained formulation is a simplification of the multi-resource constraint where only two resources need to be allocated (Dhiflaoui et al. 2018). While this is a less general formulation, since it is dedicated to a specific scenario where there are two limiting resources, it can leverage the structure of the problem and have a simpler model. This can be especially impactful in computational time for large problems since multi-resource formulations are often very complex. As is the case of the work here presented, it is often used when both workers and machines are limiting factors (Scholz-reiter et al. 2009; Lei and Guo 2014; Martins et al. 2020). However, due to the formulation's specificity, its literature is less mature.

The scheduling problem in semiconductor manufacturing is a common application in the literature. In some cases, the authors consider the need for the use of photo mask resources in addition to machines (Cakici and Mason 2007; Bitar et al. 2016), in other cases, skilled operators are required for the operation on the machines (Scholz-reiter et al. 2009).

In Cakici and Mason (2007), the environment considered is identical parallel machines, as jobs can be processed in any machine, additionally each job can only be done with one specific mask resource. The authors propose a heuristic that sorts jobs regarding their weight and inverse processing time, using the TS metaheuristic to improve results and achieve solutions close to the optimal.

In Bitar et al. (2016), the semiconductor problem is generalised and the authors consider that machines have specific capabilities, only being able to deal with a limited set of jobs. Additionally, setup times are considered to be sequence dependent. The authors propose a memetic algorithm to deal with the problem.

In Scholz-reiter et al. (2009), the authors treat the semiconductor scheduling problem as a DRC FJSP that requires the allocation of both workers and machines. The scheduling environment considers that all jobs have the same path in the job shop and workers are required for setup and disassembly. The authors propose the use of a two-level heuristic to iteratively allocate machines and operators according to sorting rules. The sequencing rules considered are First In First Out (FIFO), First in System First Out (FSFO), random and Shortest Processing Time (SPT). The presented results show that the heuristic is well suited to deal with this kind of problem, achieving results close to optimal for small instances.

In Lei and Guo (2014), the general DRC FJSP for minimisation of makespan is approached with the use of a Variable Neighbourhood Search (VNS) algorithm. The

environment considers that workers are required for the full duration of job processing. No comparison with an exact approach or lower bound is presented, complicating the performance evaluation.

In El Maraghy et al. (2000), a GA is proposed to deal with the general DRC FJSP. The GA works on the space of assigning the jobs to operators and machines while a heuristic is used for sequencing. The dispatching rules are SPT, Earliest Due Date (EDD), FIFO and Longest Processing Time (LPT). The authors develop experiments for different worker flexibilities (the higher the flexibility, the higher the number of tests a worker is suitable to process) and worker/machine balances.

Ciro et al. (2015) propose a MILP formulation for the DRC Open Shop Problem (OPP), where workers are required for the full duration of a job processing in a machine. The authors propose an Ant Colony Optimization algorithm to obtain good solutions in acceptable computational times, which, when compared to an exact method, shows promise.

Kress et al. (2019) propose a decomposition-based solution approach to deal with the DRC FJSP with sequence dependent setup times, dividing the problem into machine assignment and operation assignment. Additionally, they also propose an heuristic that reduces problem size and makes use of branch-and-cut search to solve the smaller problems.

To the best of our knowledge, only a few other papers consider partial resource occupation. The MILP formulation proposed by Cunha et al. (2019) is the first DRC FJSP formulation for the QC lab scheduling problem. A MILP model is used to obtain optimal solutions but is shown to be ineffective when dealing with instances of size that mimics the real world problem. This study deals with minimising makespan, but also considers that workers may be required at different time points in the processing of each operation. It shows that an exact solution method based on MILP is useful for small instances but is not able to obtain good quality feasible solutions for bigger instances.

The formulations in Ruiz-Torres et al. (2012, 2017), which also aims at scheduling QC labs, models the problem only considering one limiting resource, the workers. Outside of the scope of the QC lab scheduling problem, the MILP proposed by Ciro et al. (2015) presents some similarities but is less general and cannot be applied to the QC labs scheduling problem, it considers workers that are always required for the full duration of an operation.

3 Problem formulation

3.1 Quality control laboratory scheduling problem

The QC lab scheduling problem can be summarised as follows: assign each operation (test) of each job (sample) to a machine (analytical instrument) and worker (analyst/chemist), such as to minimise the total completion time.

Each operation can only be processed by predetermined subsets of machines and workers. The machine is required for the full processing time of the test while the worker is required at a number of predetermined intervals (tasks) during the pro-

cessing (e.g. machine setup, preparation of sample and materials, data processing). Workers can switch between tests as long as they are present when they are required by operations they are allocated. All tasks in an operation must be carried out by the same worker.

Each sample is an instance of a sample type, each represents a set of tests that is required to be done on a sample of a certain pharmaceutical product at a specific step in the manufacturing process. From this point on, the language that is most common in the scheduling literature is adopted: jobs, operations, machines and workers instead of samples, tests, instruments and analysts.

3.2 Mathematical model

An instance of the QC labs scheduling problem consists of a set of jobs $J = \{J_1, ...J_n\}$ where n is the total number of jobs, the set of machines $K = \{K_1, ..., K_m\}$, the set of workers $W = \{W_1, ..., W_w\}$, the set of operations $O_j = \{O_{1j}, ..., O_{q_j j}\}$, for each job $j \in J$, and the set of worker interventions $N_{ij} = \{N_{1j}, ..., N_{q_j j}\}$ for each operation $i \in O_j$ and job $j \in J$. The first three sets are unordered while the latter two are ordered. The number of operations in job j is q_j. $K_{ij} \subset K$ and $W_{ij} \subset W$ are, respectively, the subsets of machines and workers that can process operation $i \in O_j$ of job j. Each operation is characterised by its processing time p_{ij} and the start and duration times for the worker interventions, respectively, ρ^s_{ijs} and ρ^d_{ijs}. Both the start time and the duration of individual worker tasks are fixed in relation to the start time of the operation. The machine processing time is always greater than the sum of the duration of the worker tasks.

The main decision variables are the start time of operation i of job j, $t_{ij} \in \Re$, the binary assignment of operations to machines, $x_{ijk} \in \{0, 1\}$ and the binary assignment of operations to workers, $\alpha_{ijh} \in \{0, 1\}$. Additionally, based on the formulation for the FJSP with sequence-dependent setup times of Shen et al. (2018), the following sequencing variables are used: $\beta_{iji'j'} \in \{0, 1\}$ is equal to 1 if O_{ij} is scheduled before $O_{i'j'}$, 0 otherwise; and $\gamma_{ijsi'j's'} \in \{0, 1\}$ is equal to 1 if task (worker intervention) s of O_{ij} is scheduled before task s' of $O_{i'j'}$, 0 otherwise. If operation i_j and $i'_{j'}$ do not start at the same time, $\beta_{iji'j'}$ and $\beta_{i'j'ij}$ will always have different values: if operation i_j starts before $i'_{j'}$, $\beta_{i'j'ij}$ will be 1 and $\beta_{iji'j'}$ will be 0. If operation i_j starts after $i'_{j'}$, it is the other way around. However, if both operations start at the same time, both $\beta_{iji'j'}$ and $\beta_{i'j'ij}$ will be zero. The same is true for $\gamma_{ijsi'j's'}$ and $\gamma_{i'j's'ijs}$. The variable c_j is used to define the completion time of job j, this is, the time to finish all operations of that job. Finally, the variable $\mathcal{J}$ is used to define the objective function, total completion time, which is the sum of all c_j. The complete nomenclature can be found in Table 1.

Table 1 Nomenclature

Indices	
j	Subscript for jobs
i	Subscript for operations
k	Subscript for machines
h	Subscript for workers
s	Subscript for worker intervention
Dimensions	
n	Number of jobs
q_j	Number of operations of job j
m	Number of machines
w	Number of workers
l	Number of job types
Sets	
J	Set of jobs, $J = \{J_1, ..., J_n\}$
K	Set of machines, $K = \{K_1, ..., K_m\}$
W	Set of workers, $W = \{W_1, ..., W_w\}$
O_j	Set of operations of job j, $O_j = \{O_{1j}, ..., O_{q_j j}\}$
N_{ij}	Set of worker interventions s in operation O_{ij}, $N_{ij} = \{N_{1j}, ..., N_{q_j j}\}$
$K_{ij} \subset K$	Set of machines that can be used for operation O_{ij}
$W_{ij} \subset W$	Set of workers that can carry out operation O_{ij}
Parameters	
p_{ij}	Processing time of operation i of job j
ρ^s_{ijs}	Time after the start of processing of operation i of job j that intervention s is requested
ρ^d_{ijs}	Duration of intervention s in operation i of job j
M	large number
Variables	
t_{ij}	Start time of operation i of job j
x_{ijk}	$= \begin{cases} 1, \text{ if operation } i \text{ of job } j \text{ assigned to machine } k \\ 0, \text{ otherwise} \end{cases}$
α_{ijh}	$= \begin{cases} 1, \text{ if operation } i \text{ of job } j \text{ assigned to worker } h \\ 0, \text{ otherwise} \end{cases}$
$\beta_{iji'j'}$	$= \begin{cases} 1, \text{ if } O_{ij} \text{ is scheduled before } O_{i'j'} \\ 0, \text{ otherwise} \end{cases}$
$\gamma_{ijsi'j's'}$	$= \begin{cases} 1, \text{ if } O_{ij} \text{ and task } s \text{ is scheduled before} \\ \quad O_{i'j'} \text{ and task } s' \\ 0, \text{ otherwise} \end{cases}$
c_j	Completion time of job j
$\mathcal{J}$	Total completion time

Objective function

The objective of the problem is to minimise the total completion time $\mathcal{J}$,

$$\min \mathcal{J} = \sum_{j \in J} c_j \tag{1}$$

Constraints

Regarding the completion time of each job, c_j must be greater or equal than the completion time of the last operation of that job, according to constraint (2). Constraints (3) and (4) ensure that each test is assigned to one and only one of the suitable machines and workers. Constraint (5) guarantees the precedence between operations of the same job.

Constraints (6)-(7) deal with the sequencing of operations sharing machine resources while constraints (8)-(9) deal with worker intervention sequencing. Constraint (6), sequencing of any two distinct operations ($O_{ij} \neq O_{i'j'}$) in the same machine ($x_{ijk} = x_{i'j'k}$), ensures that the start time (t_{ij}) of any O_{ij} is greater or equal than the finish time ($t_{i'j'} + p_{i'j'}$) of any other $O_{i'j'}$ that is scheduled before ($\beta_{iji'j'} = 0$). As the prior constraint is disjunctive, the big M formulation is adopted. Thus, this constrain is only relevant when both operations are processed in the same machine k ($x_{ijk} = x_{i'j'k} = 1$) and $\beta_{iji'j'} = 0$, removing the impact of M ($2 - x_{ijk} - x_{i'j'k} + \beta_{iji'j'} = 0$). Similarly, constraint (7) guarantees that any O_{ij} finishes prior to any other $O_{i'j'}$ when scheduled before ($\beta_{iji'j'} = 1$) in the same machine ($x_{ijk} = x_{i'j'k} = 1$). Both constraints are required to bound for $\beta_{iji'j'} = 0$ and $\beta_{iji'j'} = 1$.

In the same way explained above, constraints (8) and (9) ensure that there are no overlapping tasks for a worker h ($\alpha_{ijh} = \alpha_{i'j'h} = 1$). Constraint (8) ensures that the start time ($t_{ij} + \rho^s_{ijs}$) of a worker intervention task s is always bigger than the end time ($t_{i'j'} + \rho^s_{i'j's'} + \rho^d_{i'j's'}$) of a preceding task ($\gamma_{ijsi'j's'} = 0$). As before, two constraints are required to bound for $\gamma_{ijsi'j'} = 0$ and $\gamma_{ijsi'j'} = 1$.

$$c_j \geq t_{q_j j} + p_{q_j j}, \quad \forall\ j \in J \tag{2}$$

$$\sum_{k \in K} x_{ijk} = 1, \quad \forall\ j \in J,\ i \in O_j \tag{3}$$

$$\sum_{h \in W} \alpha_{ijh} = 1, \quad \forall\ j \in J,\ i \in O_j \tag{4}$$

$$t_{ij} \geq t_{(i-1)j} + p_{(i-1)j}, \quad \forall\ j \in J, \quad i \in O_j, \quad i \neq 1 \tag{5}$$

$$\begin{aligned} &t_{ij} \geq t_{i'j'} + p_{i'j'} - (2 - x_{ijk} - x_{i'j'k} + \beta_{iji'j'})M, \\ &\forall\ j, j' \in J, \quad i \in O_j, \quad i' \in O_{j'}, \quad O_{ij} \neq O_{i'j'}, \quad k \in K_{ij} \cap K_{i'j'} \end{aligned} \tag{6}$$

$$\begin{aligned} &t_{i'j'} \geq t_{ij} + p_{ij} - (3 - x_{ijk} - x_{i'j'k} - \beta_{iji'j'})M, \\ &\forall\ j, j' \in J, \quad i \in O_j, \quad i' \in O_{j'}, \quad O_{ij} \neq O_{i'j'}, \quad k \in K_{ij} \cap K_{i'j'} \end{aligned} \tag{7}$$

$$t_{ij} + \rho^s_{ijs} \geq t_{i'j'} + \rho^s_{i'j's'} + \rho^d_{i'j's'} - (2 - \alpha_{ijh} - \alpha_{i'j'h} + \gamma_{ijsi'j's'})M,$$

$$\forall\ j, j' \in J,\quad i \in O_j,\quad i' \in O_{j'},\quad O_{ij} \neq O_{i'j'},\quad s, s' \in N_{ij},\quad h \in W_{ij} \quad (8)$$

$$t_{i'j'} + \rho^s_{i'j's'} \geq t_{ij} + \rho^s_{ijs} + \rho^d_{ijs} - (3 - \alpha_{ijh} - \alpha_{i'j'h} - \gamma_{ijsi'j's'})M,$$

$$\forall\ j, j' \in J,\quad i \in O_j,\quad i' \in O_{j'},\quad O_{ij} \neq O_{i'j'},\quad s, s' \in N_{ij},\quad h \in W_{ij} \quad (9)$$

3.3 Literature alternatives

The two closest existing formulations from the literature are from Shen et al. (2018) and Cunha et al. (2019). The proposed objective function is distinct from both references, minimizing total completion time instead of makespan, which it is more appropriate to the studied problem since it increases processing rate. The first reference is for one resource only, while the presented formulation is dual resource constrained. For the latter reference, the major change is the definition of auxiliary variables β and γ that identify the earliest start time of every two operations or worker interventions, respectively. These variables are fully dependant on t_{ij}, the start time of operation variable, and change the formulation considerably since the time point events (there is a time point every time there is a change in the resource allocation) can be removed from the constraints. This change results in simplified resource constraints, simplified bounding constraints, and removes all variables that defined worker intervention start and end time. The constraints that dealt with these variables, in particular the ones using big M notation, are also merged into a smaller number of constraints.

4 Three level dynamic heuristic

4.1 Scheduling approaches

The problem dimension in QC labs scheduling is an important factor to consider when developing a solution. It has been pointed out in the literature that weekly activities can amount to the tens of thousands (Maslaton 2012), resulting in a search space with a dimension that exact methods may not be suitable for. Considering how QC laboratory planning can impact the overall supply chain (Costigliola et al. 2017), a minimum viable scheduling horizon of one week is considered. Based on the QC labs environment, there is a weekly number of 70 jobs amounting to several hundred tasks.

As lab managers require the ability to generate schedules in a reduced amount of time, possibly needing to reschedule at least one time each day, the use of a heuristic approach may bring great value to improve decision making and enable its automation. Additionally, a heuristic approach is well accepted by lab managers as it can easily include new specific constraints which can vary a lot between laboratories. Some laboratories wish to ensure water based tests are done first, that some machines are used before others, allow idle times for some types of machines, etc.

The most common scheduling heuristic (Pinedo 2016; El Maraghy et al. 2000; Ruiz-Torres et al. 2012; Scholz-reiter et al. 2009; Panwalkar and Iskander 1977) consists in sorting jobs according to some characteristic (e.g. processing time) and then iteratively assigning them to machines according to some criteria (e.g. first available). This type

of approach is normally referred to as list scheduling heuristic or job dispatching rules and is also used as the basis for more advanced metaheuristic approaches such as the ones presented by Ciro et al. (2015) and Paksi and Ma'Ruf (2016).

4.2 Dynamic heuristic for the QC labs scheduling problem

This paper proposes a three-level dynamic heuristic (DH) that makes use of scheduling lists at three different levels: operation, machine and worker. Algorithm 1 presents a simplified pseudo-code for the DH. Resources (machines and workers) are chosen based on dynamic lists that take into account individual resource availability after each scheduling decision. The DH starts by virtually allocating machines to all operations (steps 4-7) based on a machine selection criteria (e.g. earliest start time). The operation list is then created and an operation is chosen (step 8) based on an operation selection criteria (e.g. smallest queue). Finally, the worker list is created for the selected operation (steps 9-10) and one is chosen (step 11) based on a worker selection criteria (e.g. earliest start time). This process is repeated until all operations are scheduled (steps 3-11).

Algorithm 1: DYNAMIC HEURISTIC

```
1  input : DRC FJSP instance
2  output : schedule
3  while list of operations is not empty do
4      for each operation do
5          for each machine that can carry out the operation do
6              compute earliest start time and finish time for carrying out the
               operation (taking into account already scheduled work)
7          attribute candidate machine to operation (with replacement)
8      choose operation based on operation selection criteria
9      for each worker that can carry out the operation do
10         compute earliest start time and finish time for carrying out the operation
           (taking into account already scheduled work)
11     choose worker based on machine selection criteria update schedule and
       remove chosen scheduled operation from list
```

The DH improves upon common static list scheduling heuristics by taking into account the dynamically computed availability of machines to check when an operation can be started and finished. For example, one level static list scheduling heuristics will sort operations for allocation based on their total processing time, without taking into account when there is availability for a machine to process it. The DH allows the operations to be also sorted based on their earliest start time or queue size of suitable machines, taking into account the availability of machines considering the currently scheduled operations. The DH also takes into account which machines and workers have the ability to carry out each operation, always generating feasible solutions. A significant amount of implementation complexity comes from assuring that constraints on operation precedence and non-overlapping tasks are met. Heuristic procedures

can create good solutions within reasonable computational time (Saidi-Mehrabad and Fattahi 2007), making them a good candidate for quality initial solutions. This is especially true for larger instances.

The virtual allocation is done for machines as they are considered the most limiting resources and their processing times are significantly greater than the ones for workers. Additionally, workers have the ability to switch between different operations when they are not required for a task. This allows them to have greater flexibility to work on operations based on the availability of the machines. This is also aligned with the method followed at quality control laboratories for manual scheduling, where machines are first scheduled and personnel is then allocated to the operations.

Possible list scheduling rules for each one of the levels in the DH are presented next. M-LAST and M-LQ result in dynamic list scheduling, other options result in a static heuristic.

List scheduling rules for machines:

- LAST: Least Available Starting Time, consisting of assigning the selected operation to the machine that can start the earliest;
- LQ: Least Queue, consisting of assigning the selected operation to the machine with the least queue.

List scheduling rules for operations:

- M-LAST: Machine Least Available Starting Time, consisting of scheduling first operations that can be started the earliest, dynamically taking into account the current already scheduled operations and the availability of machines;
- M-LQ: Machine Least Queue, consisting of scheduling first operations for which a suitable machine has the least amount of possible remaining operations to schedule, favouring early scheduling of operations with fewer suitable machines;
- SPT: Shortest Processing Time, non dynamic rule consisting of first scheduling operations with the lowest processing time first;
- LPT: Longest Processing Time, non dynamic rule consisting of first scheduling operations with the longest processing time first.

List scheduling rules for workers:

- LAFT: Least Available Finish Time, consisting of assigning the worker to the operation and machine that can finish it the earliest.
- LQ: Least Available Queue, consisting of assigning the worker with lowest queue to the operation and machine selected.

Each list is sorted using multiple rules, giving some control on how to break ties. The four heuristic configurations used are given in Table 2.

When compared to current literature, the proposed DH extends the type of heuristic proposed by El Maraghy et al. (2000) for the DRC FJSP, which is used together with a genetic algorithm, by considering dynamic rules for allocation that iteratively take into

Table 2 Sorting rules applied to each list for the four proposed heuristic configurations, given in order of priority

Configuration	Operations	Machines	Workers
M-LAST	M-LAST, SPT, M-LQ	LAST, LQ	LAFT, LQ
M-LQ	M-LQ, M-LAST, LPT	LQ, LAST	LQ, LAFT
LPT	LPT, M-LAST, M-LQ	LAST, LQ	LAFT, LQ
SPT	SPT, M-LAST, M-LQ	LAST, LQ	LAFT, LQ

account the state of the schedule. In comparison to the heuristic for QC labs scheduling proposed by Ruiz-Torres et al. (2012), the approach presented in this paper not only consider workers, but also takes into account machine resources and dynamic rules associated to them.

5 Tabu Search

Tabu Search (TS) is a metaheuristic that performs a neighbourhood search on existing solutions, which was shown to be competitive with other metaheuristic methods for large-sized problems (Akbar and Irohara 2020). TS enhances local search methods by allowing moves that will lead to worse solutions to escape local minima in the long-run. It also temporarily memorizes recent solutions to avoid revisiting them in the short-term.

The presented TS uses swap actions to move tasks between resources, either machines or workers, or to change the start order of operations. Machine swaps move all three tasks of the same operation at once since these must be done by the same machine resource. For the order swap operation, two tasks on the same resource are randomly selected and their position is switched.

The Tabu Search implementation is described in Algorithm 2. Starting from a given initial solution (step 3), TS will generate a fixed number of candidate solutions by applying different swap operations to the initial solution (*inner loop*, steps 5-6). The best candidate solution (found in steps 7-10) will be used as starting point of next inner loop and is recorded in a *tabu list* (step 13). The *outer loop* (steps 4-13) is repeated until any stopping criteria are met.

Candidate solutions already on the tabu list are never selected as best candidate solution (step 9), forcing new schedules to be exploited. Since the tabu list has a size limit, eventually all solutions are replaced, allowing older tabu listed solutions to be exploited again. Every outer loop, the global best solution is also updated if a better candidate solution is found (steps 11-12).

The parameters used for the tabu search are: 200 outer loops (steps 4-13); 50 candidate solutions generated in the inner loop (steps 5-6); tabu list holds at maximum 20 solutions; maximum of one hour run time.

To simplify the swapping operations, the schedule solutions are encoded into resource lists, lowering the computational cost and simplifying the generation of new feasible solutions. This is specially suited for neighbourhood search methods such as

Algorithm 2: TABU SEARCH

```
1  input : initial schedule solution
2  output : best schedule solution found
3  define best candidate as initial solution
4  while time limit not reached do
5      for tabu search inner loop step do
6          generate a new candidate solution from the best candidate by randomly swapping either
           machine or worker tasks
7      define best candidate solution as the first candidate solution
8      for generated candidate solutions do
9          if candidate solution better than best candidate solution and not in tabu list then
10             update best candidate solution with candidate solution
11     if best candidate better than global best then
12         update global best
13     append best candidate solution to tabu list
```

local search. A decoded solution is a list where all the relevant schedule information is detailed for each task: job, operation, resource, task, start time, end time, resource type (machine or worker) and resource index. The encoded solution is a list of jobs per resource. Each list holds job-operation pairs, where each duplicate pair represents one of the three operation tasks, which are assumed to always be in order.

The decoder routines works by iteratively selecting the tasks from the lists and assigning them the feasible leftmost position. Tasks are selected by earliest start time and lowest queue position, considering the maximum of either resource and only considering tasks that do not violate precedence constraints. While this greatly simplifies the encoded solution, some feasible solutions can no longer be represented. However, by always moving all tasks to the earliest time feasible, some encoded solutions have better total completion time than the original decoded solutions.

6 Experimental results

A set of experiments is designed in order to test the proposed formulation and evaluate the performance of the proposed heuristic. These experiments are representative of a real QC laboratory and parameters such as processing times, number of resources and volume of jobs are in the same order of magnitude of the original values from the real case study (Costigliola et al. 2017; Lopes et al. 2020). While real QC laboratories can deal with many different types of analysis and machines, the bulk of the workload is often a smaller subset of popular, recurring analysis. Based on the original QC unit, two base scenarios are emulated: daily workload (10 jobs) and weekly workload (70 jobs). Each experiment characterises a different configuration of QC labs, while multiple instances of each experiment are generated to improve the reliability of results.

To evaluate the quality of the solutions, an exact method was also used to solve the mathematical model. For this, the industrial combinatorial optimisation software CPLEX was applied. CPLEX solves the MILP problem using a branch-and-bound algorithm.

The Tabu Search described in the previous section is also used to evaluate the performance of the heuristics, evaluating both a pure TS approach and a hybrid TS with initial solution given by an heuristic method.

6.1 Description of experiments

The parameters that characterise each experiment are the number of jobs n, number of machines m, number of workers w and flexibility f. The presented experiments are developed for n equal to 5, 10 and 70. The smaller 5 job instances are used to evaluate the performance of the heuristic in comparison to the optimal solutions obtained by CPLEX. The medium instances of 10 jobs represent the daily workload of a QC laboratory. The larger 70 job instances mimic the realistic QC labs weekly scheduling problem. The number of machines is set at 7, representing different types of equipment present at laboratories. Number of workers can be 2, 3 and 7, representing cases that are, respectively, worker restricted, balanced and machine restricted.

The experiments developed encompass different configurations of the QC labs scheduling problem. Regarding the generation of jobs for each experiment, distributions based on the experiments of Malve and Uzsoy (2007) and Ruiz-Torres et al. (2012) are used. The instance generation approach followed makes use of the sample type (job type) concept, which has direct relation to what may happen in real QC labs. Each combination of product and source (e.g. raw material, final product) usually results on a predetermined set of tests (operations) that need to be done. This way, jobs come from a set of predetermined possible job types, each characterised by a number of operations and their processing times.

Regarding the job types, l, a total of 3 are considered in all experiments. For each job type, the number of operations is determined using a discrete uniform distribution (DU) ranging from 1 to 3. Each operation has a processing time p_{ij} ranging from 1 to 5 and the worker tasks start from three time points: the start of the operation, at 30% of operation processing and at 90% of operation processing. The durations of the worker tasks are: 5%, 10% and 10% of processing time, representing the setup, intermediate and disassembly/data processing tasks.

The flexibility parameter f is used to compute the machines and workers that can carry out a certain operation. On QC laboratories, each job is a unique combination of analytical tests and respective necessary machines. This means that only specific machines can perform certain tests and analysts aren't trained to do every possible analysis. To represent this, when generating the instance, each one of the workers and machines have a probability equal to the flexibility of being eligible to carry out the operation. Experiments are done for flexibilities of 30% and 60%, encompassing cases where machines and workers have less or more general competences.

For each configuration of job, worker and flexibility, three problem instances are generated (replications) to improve the reliability of results. With n taking three possible values, w taking three possible values and the f two possible values, the total number of instances with different experimental parameters is eighteen. Considering that each of these instances has three distinct unique parameter variations, this is three replications, there is a total of fifty-four distinct experiments. All the experiment

Table 3 Parameters for the experiments designed

Parameter		Values
Job generation parameters		
Job types	l	3
Operations in job	q_j	$DU(1, 3)$
Operation processing time	p_{ij}	$DU(1, 5)$
Starting point of worker tasks	ρ^s_{ijs}	$[0, 0.3p_{ij}, 0.9p_{ij}]$
Duration of worker tasks	ρ^d_{ijs}	$[0.05p_{ij}, 0.1p_{ij}, 0.1p_{ij}]$
Experiment parameters		
Number of jobs	n	[5, 10, 70]
Number of machines	m	7
Number of workers	w	[2, 3, 7]
Flexibility	f	[0.3, 0.6]

A total of eighteen parameter combinations and fifty-four total instances are used, considering that three replications are created for each parameter combination

parameters are summarised on Table 3. For further comparison, all the results are presented in one single table in appendix A, in Table 8.

CPLEX was selected as baseline for this paper, but the Gurobi solver was also tested on both the complete (GC) and the heuristic (GH) versions. For the 5 job instances, all solvers achieve the same results. For the 10 job instances GH has the best performance (finding the best solution in 15 out of 18 experiments), followed by the GC (10 out of 18), in turn closely followed by CPLEX (9 out of 18). The average relative error to the best solution found for each experiment is: CPLEX 1.3%, GC 0.7%, GH 0.4%. These results can be seen on Appendix A. However, Gurobi is not able to achieve a feasible solution for any 70-job instance in the 12-hour limit. Thus, the CPLEX solver is used as baseline in this paper. Note that, for the 10-job instances, the difference in performance of exact methods is small enough that choosing CPLEX does not change the conclusions regarding comparisons with all other methods detailed in this paper.

The dynamic heuristic approaches are deterministic. The CPLEX parameter configuration does not use randomization. Namely, the used search type is *Restart*, making the used MILP version deterministic. Thus, both the heuristic and the MILP approaches run once per problem instance. The presented Tabu Search results are an average of ten different runs, with the corresponding standard deviation presented as well. To the small and medium instances (5 and 10 jobs), all approaches have a maximum run time of one hour. For the large instances (70 jobs), this limit is twelve hours for CPLEX and one hour for TS.

All experiments ran on a computer with Windows 10 and an Intel Core i7-3930K processor (3.2GHz base frequency) and 32 gigabytes of DDR4 RAM. CPLEX 12.10 is used through its Python API and the heuristics are implemented in Python.[1]

[1] Source code can be found in the repository https://github.com/miguelsem/Minimizing-total-completion-time-in-large-sized-pharmaceutical-quality-control-scheduling.

6.2 Heuristic rules comparison

Results for all experiments with each heuristic configuration, as detailed in Table 2, are presented in Table 4. To evaluate the quality of the heuristics, result and lower bound found by the CPLEX solver are also presented.

For 5 and 10 job instances, M-LAST and M-LQ give the best heuristic performances at almost all experiments, but are always outperformed or matched by the exact method. However, for the 70 job instances, M-LAST is by far the best heuristic approach and the exact method can never compete with it. This is a consequence of the exponentially growing complexity of the MILP formulation, as can be seen in Table 5.

For the small instances with $n = 5$, M-LAST and M-LQ often show the best performance. The CPLEX is almost always able to obtain the optimal solution below the target 1% gap in a median time of ten seconds,[2] excluding one outlier. For comparison, the dynamic heuristic generates solutions for these instances in a median time of two seconds.

For the medium instances with $n = 10$, M-LAST shows a clear performance improvement comparing with other heuristics. While the exact method still provides the best results overall, it rarely achieves the optimal solution at this complexity. Considering a time limit of one computation hour and a minimum gap of 1%, all experiments but two were stopped before finding the optimal solution. For comparison, the DH generates solutions for these instances in a median time of 6 seconds.

For the large instances with $n = 70$, the M-LAST configuration almost always leads to the best result, even in comparison with the exact method. Thus, M-LAST is the single heuristic that will be compared with the remaining strategies. Regarding CPLEX results, considering a time limit of twelve hours of computation, no solutions within the target 5% gap were achieved. Furthermore, for the three final instances no feasible solutions could be found by CPLEX. For comparison, the DH generates solutions for these instances in a median time of five minutes. As expected, it is also visible that higher flexibility instances show worse performance of the exact method, as can be seen by the large difference between upper and lower bounds.

6.3 Tabu search

Tabu Search results are summarised in Table 6. For each experiment, the average total completion time of ten runs is presented for two Tabu Search scenarios: starting from a random solution and for using the dynamic heuristic M-LAST as a starting solution. Due to the simplicity of the small 5 job instances, these are not shown here. The average value is also accompanied by the standard deviation.

For the 10-job instances the Tabu Search approach using a random initial solution gives better results than the heuristic or, less often, worse but competitive results. However, for the 70-job instances the results are not as close and the Tabu Search can only surpass the heuristic 4 times. Furthermore, using the heuristic solution as initial solution of the Tabu Search approach reaches good results for all job sizes, even if this

[2] Increasing the complexity to 8 jobs would already increase this median time to 47 minutes, only arriving at an optimal solution within one hour of computation time for half of the experiments

Table 4 Total completion times (h) of the four different sets of sorting rules and respective CPLEX solutions

n	*w*	*f*	*rep*	M-LAST	M-LQ	SPT	LPT	CPLEX	
								Upper	Lower
5	2	0.3	0	43.8	*43.4*	44.8	49.4	**41.3***	41.3
			1	*62.6*	63.9	*62.6*	64.8	**54.0***	54.0
			2	*25.7*	26.4	29.0	26.4	**23.6***	23.6
		0.6	0	*46.9*	47.9	50.4	50.2	**41.8***	41.8
			1	*49.3*	53.9	50.6	56.1	**44.9**	44.5
			2	*23.6*	*23.6*	24.0	24.2	**23.4***	23.4
	3	0.3	0	74.3	*74.2*	82.8	74.6	**69.9***	69.9
			1	35.5	*34.5*	35.6	*34.5*	**34.3***	34.3
			2	49.6	*44.2*	51.3	49.6	**43.2***	43.2
		0.6	0	*28.7*	29.7	*28.7*	29.7	**27.4***	27.4
			1	*28.0*	33.0	28.8	32.8	**26.7***	26.7
			2	34.7	*34.6*	38.1	*34.6*	**33.0***	33.0
	7	0.3	0	*30.5*	37.0	37.3	44.5	**29.7***	29.7
			1	48.2	*46.8*	50.7	47.4	**46.4***	46.4
			2	25.3	27.2	*25.2*	28.0	**24.1***	24.1
		0.6	0	*27.0*	27.3	*27.0*	28.0	**26.0***	26.0
			1	46.1	*44.2*	47.1	48.0	**42.1***	42.1
			2	**27.0***	**27.0***	**27.0***	**27.0***	**27.0***	27.0
10	2	0.3	0	*101.4*	124.6	107.5	109.9	**90.7**	85.3
			1	244.2	*218.4*	296.9	235.8	**189.5**	109.9
			2	*58.6*	72.3	71.1	74.7	**54.8**	43.0
		0.6	0	*143.6*	151.8	146.2	148.4	**125.6**	90.5
			1	*112.7*	124.2	126.1	143.2	**111.3**	80.5
			2	*130.7*	147.5	133.8	159.8	**129.3**	125.5
	3	0.3	0	126.4	*124.7*	134.6	138.1	**108.1***	108.1
			1	*133.7*	164.7	152.4	194.5	**126.9**	90.9
			2	108.6	*106.8*	113.4	133.7	**96.0**	91.3
		0.6	0	77.5	75.7	84.2	*73.8*	**59.9**	55.2
			1	100.1	*94.1*	110.6	109.1	**90.6**	66.0
			2	83.3	99.4	*77.8*	98.5	**71.2**	62.6
	7	0.3	0	*112.8*	119.8	117.6	121.6	**102.5***	102.5
			1	*120.3*	124.9	121.5	141.0	**119.5**	117.0
			2	*93.3*	112.3	108.5	124.0	**93.0**	82.0
		0.6	0	*108.4*	116.7	122.4	131.0	**99.3**	96.1
			1	*97.6*	122.0	113.1	121.1	**97.1**	80.0
			2	107.4	91.3	125.0	*90.2*	**82.2**	79.0

Table 4 continued

n	w	f	rep	M-LAST	M-LQ	SPT	LPT	CPLEX	
								Upper	Lower
70	2	0.3	0	**3930.7**	5684.0	4848.7	4995.6	8253.2	1416.6
			1	**5265.0**	7966.4	6129.6	7502.2	9966.9	713.5
			2	3924.0	4182.8	4834.1	**3855.1**	7245.8	644.0
		0.6	0	**4075.5**	5084.1	4878.1	5492.3	10356.3	568.4
			1	**4374.6**	5701.8	5488.3	5047.6	11392.8	671.3
			2	**3955.3**	5756.7	4189.7	5956.9	9084.5	697.5
	3	0.3	0	**4183.0**	5808.8	4384.6	4457.6	8627.4	611.5
			1	**8318.2**	8450.8	10034.0	8904.9	13935.7	4959.0
			2	**4442.4**	6345.1	5511.1	6458.8	6369.5	1076.0
		0.6	0	**2245.0**	3602.5	2790.3	3496.1	6528.2	510.2
			1	**3262.9**	3430.4	3838.4	3890.5	8161.3	613.5
			2	**1849.5**	2507.8	2270.4	2891.9	2916.5	435.1
	7	0.3	0	**3455.5**	3516.0	3499.2	4121.9	8078.1	1817.0
			1	4630.2	**4374.2**	4977.1	4754.0	9783.2	2744.2
			2	**2760.9**	2978.4	3017.9	2969.4	6068.6	1771.0
		0.6	0	**3352.9**	4658.9	3774.8	4642.0	–	581†
			1	**1836.3**	2537.1	2146.1	2844.5	–	374†
			2	**2377.1**	3618.7	3159.9	3900.3	–	508†

CPLEX columns also details if: optimal solution found (*); only feasible solution found is LP relaxation of root node (†). Each row highlights the best solution (bold) and the best heuristic (italic)

Table 5 Comparison of model information for an instance with $n = 5$, $n = 10$ and $n = 70$

	Model information		
	$n = 5$	$n = 10$	$n = 70$
Variables	2033	6378	346780
Binary variables	2014	6343	346524
Constraints	2504	8600	595920

improvement shrinks as complexity increases. One extreme case can be seen on the final instance where TS cannot improve upon the initial solution.

To sum up, using TS on the heuristic solution is a very good approach for small to medium instances, but as complexity increases the effectiveness of the TS vanishes. These results reinforces the idea that the solutions obtained using the M-LAST heuristic are determinant to obtain good performance.

6.4 Overall comparison

Table 7 shows the comparison between the best configurations of each type of approach: heuristic (M-LAST), metaheuristic (TABU with heuristic initial solution)

Table 6 Tabu Search comparison of average total completion time (h) and standard deviation for different initial solutions

Jobs	Workers	Flex	Rep	M-LAST	TABU random	TABU M-LAST
10	2	0.3	0	101.4	94.1 ± 2.2	**93.7 ± 1.2**
			1	244.2	219.8 ± 7.7	**211.4 ± 5.3**
			2	58.6	57.8 ± 1.5	**56.7 ± 1.0**
		0.6	0	143.6	138.8 ± 4.5	**125.4 ± 1.9**
			1	112.7	114.5 ± 2.1	**109.1 ± 2.4**
			2	130.7	130.0 ± 1.8	**128.6 ± 0.3**
	3	0.3	0	126.4	111.7 ± 4.6	**111.4 ± 3.1**
			1	133.7	130.3 ± 3.3	**126.5 ± 1.4**
			2	108.6	100.1 ± 2.8	**99.2 ± 1.5**
		0.6	0	77.5	62.1 ± 1.7	**61.7 ± 1.0**
			1	100.1	93.0 ± 2.3	**91.1 ± 1.8**
			2	83.3	78.2 ± 2.7	**75.1 ± 1.8**
	7	0.3	0	112.8	111.4 ± 3.9	**108.2 ± 1.7**
			1	120.3	122.9 ± 1.4	**120.3 ± 0.1**
			2	93.3	97.9 ± 3.9	**93.1 ± 0.1**
		0.6	0	108.4	107.5 ± 3.6	**104.9 ± 2.7**
			1	97.6	102.8 ± 2.3	**97.5 ± 0.2**
			2	107.4	88.4 ± 1.9	**87.3 ± 1.7**
70	2	0.3	0	3930.7	4525.7 ± 238.1	**3709.1 ± 39.9**
			1	5265.0	7014.0 ± 220.7	**5089.2 ± 31.1**
			2	3924.0	3724.5 ± 89.1	**3616.6 ± 27.2**
		0.6	0	4075.5	4761.2 ± 131.5	**3355.4 ± 53.6**
			1	4374.6	5309.3 ± 235.0	**4157.1 ± 36.9**
			2	3955.3	5049.4 ± 189.7	**3536.2 ± 90.3**
	3	0.3	0	4183.0	4148.8 ± 204.3	**3677.7 ± 78.7**
			1	8318.2	10407.3 ± 685.5	**7110.8 ± 358.7**
			2	4442.4	4812.9 ± 142.2	**4168.0 ± 30.9**
		0.6	0	2245.0	3160.1 ± 125.9	**2240.2 ± 10.2**
			1	3262.9	3259.4 ± 104.6	**2746.1 ± 55.7**
			2	1849.5	2039.9 ± 32.9	**1803.8 ± 18.4**
	7	0.3	0	3455.5	3325.4 ± 42.2	**3323.2 ± 60.3**
			1	4630.2	4647.3 ± 165.9	**4424.8 ± 44.5**
			2	2760.9	2788.8 ± 43.1	**2704.9 ± 32.6**
		0.6	0	3352.9	4697.4 ± 70.1	**3351.3 ± 3.8**
			1	1836.3	2182.6 ± 74.0	**1819.8 ± 3.9**
			2	**2377.1**	3644.1 ± 109.9	**2377.1 ± 0.0**

Columns show solution from heuristic M-LAST, TS with random initial solution and TS with M-LAST initial solution, respectively. The best result for each instance is highlighted in bold

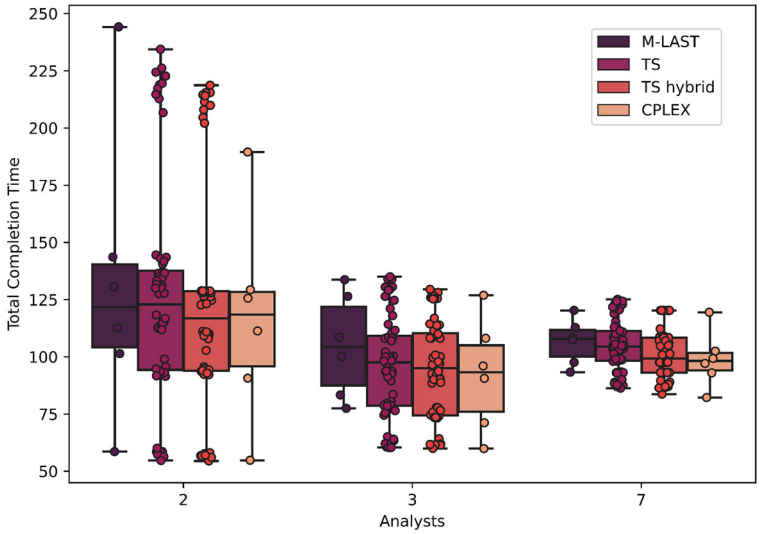


Fig. 1 Performance comparison of the different implemented approaches for changing balance between number of machines and workers for the medium 10 job instances ($n = 10$)

and exact method (CPLEX solver). Since the small 5 job instances are very simple and can be solved to optimality by CPLEX easily, these are not represented in this table.

It is noteworthy that the length of the schedule solutions, the makespan, is consistent with the original data. The makespan of the solutions is between 12-24 hours for the daily instances and 5-8 days for the weekly instances, assuring that the job dimensions and processing times are correctly dimensioned.

Day sized instances

Regarding the results for the medium-sized experiments with $n = 10$, CPLEX outperforms the other configuration for approximately 78% of the experiments. On all remaining configurations the hybrid Tabu Search approach provides the best solution. In Fig. 1, the total completion time of the solutions obtained using the heuristic methods is pictured for a growing number of workers. The range of the solutions for different analysts shows that with two workers the problem is mainly worker constrained and with seven it is mostly machine constrained. It can be seen that as more analysts are added to the problem the CPLEX solver becomes closer to the other approaches. Also, from $w = 3$ to $w = 7$ the average solution makespan is similar, possibly indicating that three workers results in a balanced case and adding additional personnel is not beneficial. Note that while the heuristic has a performance around 6.6% worse than CPLEX solver, by running so much faster, under a minute instead of under an hour, it has potential applications in rescheduling of dynamic environments. Using Tabu Search on top of the heuristic solution brings this difference of around 1.5%. In comparison, when using Tabu Search with initial random solution the distance is around 2.1%.

Table 7 Final comparison between best heuristic configuration, CPLEX upper bound and Tabu Search (with M-LAST initial solution)

Jobs	Workers	Flex	Rep	M-LAST	CPLEX	TABU + M-LAST
10	2	0.3	0	101.4	**90.7**	93.7 ± 1.2
			1	244.2	**189.5**	211.4 ± 5.3
			2	58.6	**54.8**	56.7 ± 1.0
		0.6	0	143.6	125.6	**125.4 ± 1.9**
			1	112.7	111.3	**109.1 ± 2.4**
			2	130.7	129.3	**128.6 ± 0.3**
	3	0.3	0	126.4	**108.1**	111.4 ± 3.1
			1	133.7	126.9	**126.5 ± 1.4**
			2	108.6	**96.0**	99.2 ± 1.5
		0.6	0	77.5	**59.9**	61.7 ± 1.0
			1	100.1	**90.6**	91.1 ± 1.8
			2	83.3	**71.2**	75.1 ± 1.8
	7	0.3	0	112.8	**102.5**	108.2 ± 1.7
			1	120.3	**119.5**	120.3 ± 0.1
			2	93.3	**93.0**	93.1 ± 0.1
		0.6	0	108.4	**99.3**	104.9 ± 2.7
			1	97.6	**97.1**	97.5 ± 0.2
			2	107.4	**82.2**	87.3 ± 1.7
70	2	0.3	0	3930.7	8253.2	**3709.1 ± 39.9**
			1	5265.0	9966.9	**5089.2 ± 31.1**
			2	3924.0	7245.8	**3616.6 ± 27.2**
		0.6	0	4075.5	10356.3	**3355.4 ± 53.6**
			1	4374.6	11392.8	**4157.1 ± 36.9**
			2	3955.3	9084.5	**3536.2 ± 90.3**
	3	0.3	0	4183.0	8627.4	**3677.7 ± 78.7**
			1	8318.2	13935.7	**7110.8 ± 358.7**
			2	4442.4	6369.5	**4168.0 ± 30.9**
		0.6	0	2245.0	6528.2	**2240.2 ± 10.2**
			1	3262.9	8161.3	**2746.1 ± 55.7**
			2	1849.5	2916.5	**1803.8 ± 18.4**
	7	0.3	0	3455.5	8078.1	**3323.2 ± 60.3**
			1	4630.2	9783.2	**4424.8 ± 44.5**
			2	2760.9	6068.6	**2704.9 ± 32.6**
		0.6	0	3352.9	†	**3364.2 ± 0.0**
			1	1836.3	†	**1828.2 ± 4.2**
			2	**2377.1**	†	**2452.3 ± 0.0**

Results represent minimisation of total completion time (hours). Some CPLEX solutions only returned the LP relaxation of root node (†) and are omitted from the table

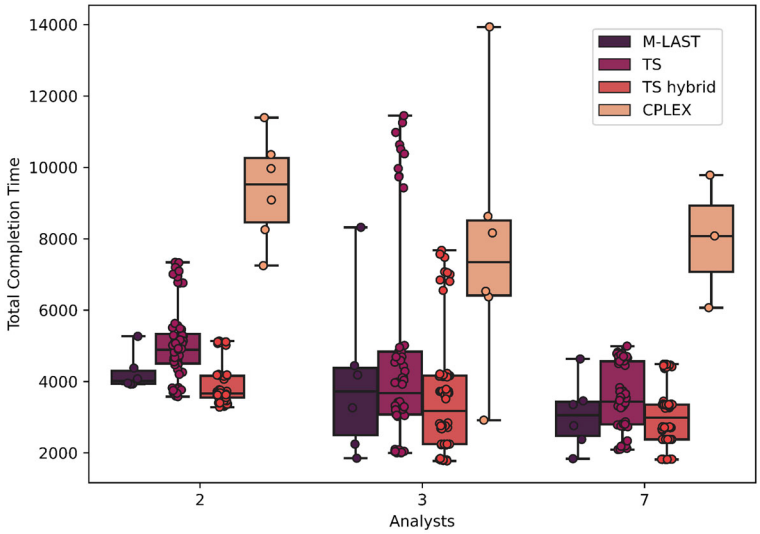


Fig. 2 Performance comparison of the different implemented approaches for changing balance between number of machines and workers for the large 70 job instances ($n = 70$)

Week sized instances

Regarding the large-sized experiments with $n = 70$, the hybrid approach Tabu Search with initial heuristic solution from M-LAST always outperformed or matched the other approaches. However, this success can be mostly attributed to the initial solution, as concluded from Table 6. The heuristic approach and both TS versions reach results with approximately 50% less total completion time. For the heuristic, this results are obtained in a fraction of the time. In Fig. 2, the total completion time for a growing number of workers is again pictured for the larger instances. The average completion time of all solutions improves from $w = 2$ to $w = 3$, indicating that the seven machines are underused with two workers. The same can be said comparing $w = 3$ and $w = 7$.

Overall, the Tabu Search improvements over the pure heuristic results are consistent. While the hybrid TS improves the initial solution by approximately 7.4% on average, the random start TS only achieves a solution 12.7% worse than heuristic. Keeping in mind the dynamic environment of a QC laboratory, the pure heuristic approach can be more useful for time-sensitive real world implementation, complementing with the hybrid Tabu Search when time is not an issue.

Overall comparison with job scaling

Figure 3 plots the total completion time for instances individually. All nine presented instances have 3 workers and 0.3 flexibility, one for each job number (5, 10, 70) and each one of the three replications.

As job complexity increases, heuristic performance becomes competitive with Tabu Search and surpasses CPLEX results. The hybrid TS, using the heuristic solution as initial solution, will always improve upon the heuristic results but this improvement

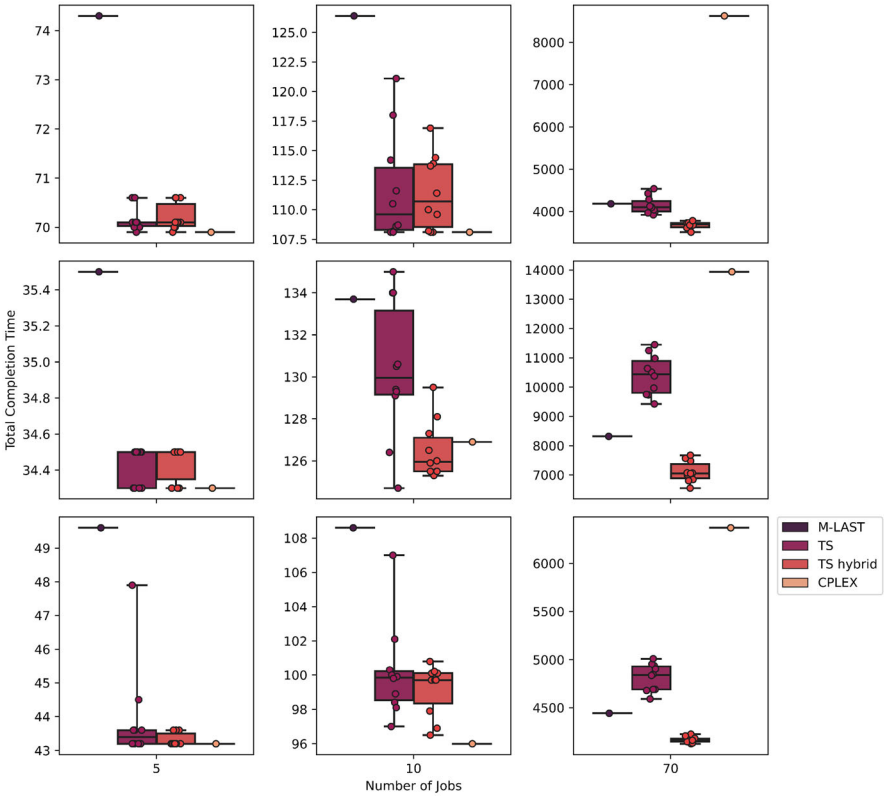


Fig. 3 Total completion time of the different implemented approaches for each individual instance with 3 workers and 0.3 flexibility

becomes smaller as complexity increases. Also, while TS and hybrid TS are competitive between themselves for smaller instances, TS hybrid will more often reach better results for more complex instances. Lastly, CPLEX goes from finding the best solutions overall to only reaching solutions with twice as much total completion time as the other approaches.

7 Conclusions

This paper proposes an heuristic approach custom made for a real world scheduling problem of quality control laboratories in the pharmaceutical industry. Management of lab operations is complex and time consuming, making tools that automatically allocate resources very valuable. The proposed algorithm is successful in obtaining good quality and useful solutions in a timely manner. If time is not an impediment, using Tabu Search as a local search procedure on the heuristic result can be beneficial.

The problem is an extension to the dual resource constrained flexible job shop scheduling problem, consisting in the allocation of both machine and worker resources to analytical chemistry laboratory jobs with the objective to minimise the total com-

pletion time. A novel mixed integer linear program formulation is presented. The proposed dynamic heuristic improves upon common list scheduling heuristics, proving to be suitable when dealing with instances of size mimicking the real problem.

Regarding the experimental results, when compared to CPLEX solver, for small instances, the proposed heuristic is not able to obtain optimum solutions and its performance is worse. On the other hand, when dealing with instances of a size that is useful for real applications, the proposed dynamic heuristic significantly outperforms CPLEX, which in multiple cases is not able to find feasible solutions. The heuristic is also significantly faster (median time of five minutes) than CPLEX (makes use of the twelve hours given to it for the larger instances). Tabu Search can always improve the heuristic solution, but its effectiveness diminishes with increasing complexity. The computational cost of the metaheuristic might be prohibitive for real world implementation given the small improvement to the heuristic result.

Dealing with the laboratory scheduling problem in the pharmaceutical industry as a dual resource constrained problem, the algorithm proposed is able to generate schedules that can directly be followed in practice. Even though this type of solution has great potential if applied in practice, multiple challenges to its implementation can be found in the current practices followed in laboratories. Managers involved in the implementation of this type of solution must be aware of requirements that must be complied to enable the use of automated solutions for resource allocation: processes must be in place to gather and maintain systematic data on laboratory workers and instruments; types of samples and analytical methods must be completely characterised, including the complete survey of test processing times; managers and team leaders must be surveyed to gather all additional constraints found in the lab environment; processes must be put in place to enable the updating of schedule due to unexpected reasons (very common in quality control laboratories) and platforms must be made available for laboratory workers to be able to input updated information on current operations and resource availability.

While already being competitive with the other approaches, the proposed heuristic shows potential to be further used as a basis of more complex optimisation strategies. The low computational cost and capacity to generate feasible solutions has been shown to be beneficial when paired with neighbourhood search methods. Furthermore, the fast heuristic run times makes obtaining a new solution from scratch a feasible way to deal with rescheduling of new jobs, machine breakdowns, and other disruptions to live deployment. The proposed heuristic can also be used to generate initial feasible solutions for a population used in a genetic algorithm, which can be difficult to apply in this type of problem due to the large search space and relativity small space of feasible solutions.

Acknowledgements We would like to thank Hovione for the problem motivation and for the integral role defining the benchmark instances presented in this paper. This work is financed by national funds through FCT - Foundation for Science and Technology, I.P., through IDMEC, under LAETA, project UIDB/50022/2020. The work of Miguel S. E. Martins was supported by the PhD Scholarship 2020.08776.BD from FCT.

Funding Open access funding provided by FCT|FCCN (b-on).

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

Appendix A: Full results table

Table 8 Total completion times (h) of all the methods

n	w	f	rep	CPLEX		Heuristics				Tabu search	
				Upper bound	Lower bound	M-LAST	M-LQ	SPT	LPT	Random start	M-LAST start
5	2	0.3	0	**41.3***	41.3	43.8	*43.4*	44.8	49.4	42.3 ± 0.6	42.3 ± 0.6
			1	**54.0***	54.0	*62.6*	63.9	*62.6*	64.8	57.9 ± 2.4	56.4 ± 1.0
			2	**23.6***	23.6	*25.7*	26.4	29.0	26.4	23.8 ± 0.2	**23.6 ± 0.0***
		0.6	0	**41.8***	41.8	*46.9*	47.9	50.4	50.2	44.6 ± 0.8	42.8 ± 0.7
			1	**44.9**	44.5	*49.3*	53.9	50.6	56.1	47.2 ± 1.5	46.6 ± 0.3
			2	**23.4***	23.4	*23.6*	*23.6*	24.0	24.2	23.6 ± 0.0	23.6 ± 0.0
	3	0.3	0	**69.9***	69.9	74.3	*74.2*	82.8	74.6	70.2 ± 0.2	70.2 ± 0.3
			1	**34.3***	34.3	35.5	*34.5*	35.6	*34.5*	34.4 ± 0.1	34.4 ± 0.1
			2	**43.2***	43.2	49.6	*44.2*	51.3	49.6	43.9 ± 1.5	43.3 ± 0.2
		0.6	0	**27.4***	27.4	*28.7*	29.7	*28.7*	29.7	27.8 ± 0.1	27.8 ± 0.0
			1	**26.7***	26.7	*28.0*	33.0	28.8	32.8	26.7 ± 0.0	**26.7 ± 0.0***
			2	**33.0***	33.0	34.7	*34.6*	38.1	*34.6*	33.4 ± 0.8	33.2 ± 0.4
	7	0.3	0	**29.7***	29.7	*30.5*	37.0	37.3	44.5	29.9 ± 0.2	29.8 ± 0.2
			1	**46.4***	46.4	48.2	*46.8*	50.7	47.4	46.7 ± 0.5	48.0 ± 0.6
			2	**24.1***	24.1	25.3	27.2	*25.2*	28.0	25.2 ± 0.6	25.1 ± 0.3
		0.6	0	**26.0***	26.0	*27.0*	27.3	*27.0*	28.0	26.3 ± 0.4	26.1 ± 0.3
			1	**42.1***	42.1	46.1	*44.2*	47.1	48.0	43.0 ± 1.7	42.8 ± 1.4
			2	**27.0***	27.0	**27.0***	**27.0***	**27.0***	**27.0***	27.0 ± 0.1	**27.0 ± 0.0***
10	2	0.3	0	**90.7**	85.3	*101.4*	124.6	107.5	109.9	94.1 ± 2.2	93.7 ± 1.2
			1	**189.5**	109.9	244.2	*218.4*	296.9	235.8	219.8 ± 7.7	211.4 ± 5.3
			2	**54.8**	43.0	*58.6*	72.3	71.1	74.7	57.8 ± 1.5	56.7 ± 1.0
		0.6	0	125.6	90.5	*143.6*	151.8	146.2	148.4	138.8 ± 4.5	**125.4 ± 1.9**
			1	111.3	80.5	*112.7*	124.2	126.1	143.2	114.5 ± 2.1	**109.1 ± 2.4**
			2	129.3	125.5	*130.7*	147.5	133.8	159.8	130.0 ± 1.8	**128.6 ± 0.3**
	3	0.3	0	**108.1***	108.1	126.4	*124.7*	134.6	138.1	111.7 ± 4.6	111.4 ± 3.1
			1	126.9	90.9	*133.7*	164.7	152.4	194.5	130.3 ± 3.3	**126.5 ± 1.4**
			2	**96.0**	91.3	108.6	*106.8*	113.4	133.7	100.1 ± 2.8	99.2 ± 1.5
		0.6	0	**59.9**	55.2	77.5	75.7	84.2	*73.8*	62.1 ± 1.7	61.7 ± 1.0
			1	**90.6**	66.0	100.1	*94.1*	110.6	109.1	93.0 ± 2.3	91.1 ± 1.8

Table 8 continued

n	w	f	rep	CPLEX		Heuristics				Tabu search	
				Upper bound	Lower bound	M-LAST	M-LQ	SPT	LPT	Random start	M-LAST start
			2	**71.2**	62.6	83.3	99.4	*77.8*	98.5	78.2 ± 2.7	75.1 ± 1.8
	7	0.3	0	**102.5***	102.5	*112.8*	119.8	117.6	121.6	111.4 ± 3.9	108.2 ± 1.7
			1	**119.5**	117.0	*120.3*	124.9	121.5	141.0	122.9 ± 1.4	120.3 ± 0.1
			2	**93.0**	82.0	*93.3*	112.3	108.5	124.0	97.9 ± 3.9	93.1 ± 0.1
		0.6	0	**99.3**	96.1	*108.4*	116.7	122.4	131.0	107.5 ± 3.6	104.9 ± 2.7
			1	**97.1**	80.0	*97.6*	122.0	113.1	121.1	102.8 ± 2.3	97.5 ± 0.2
			2	**82.2**	79.0	107.4	91.3	125.0	*90.2*	88.4 ± 1.9	87.3 ± 1.7
70	2	0.3	0	8253.2	1416.6	*3930.7*	5684.0	4848.7	4995.6	4525.7 ± 238.1	**3709.1 ± 39.9**
			1	9966.9	713.5	*5265.0*	7966.4	6129.6	7502.2	7014.0 ± 220.7	**5089.2 ± 31.1**
			2	7245.8	644.0	3924.0	4182.8	4834.1	*3855.1*	3724.5 ± 89.1	**3616.6 ± 27.2**
		0.6	0	10356.3	568.4	*4075.5*	5084.1	4878.1	5492.3	4761.2 ± 131.5	**3355.4 ± 53.6**
			1	11392.8	671.3	*4374.6*	5701.8	5488.3	5047.6	5309.3 ± 235.0	**4157.1 ± 36.9**
			2	9084.5	697.5	*3955.3*	5756.7	4189.7	5956.9	5049.4 ± 189.7	**3536.2 ± 90.3**
	3	0.3	0	8627.4	611.5	*4183.0*	5808.8	4384.6	4457.6	4148.8 ± 204.3	**3677.7 ± 78.7**
			1	13935.7	4959.0	*8318.2*	8450.8	10034.0	8904.9	10407.3 ± 685.5	**7110.8 ± 358.7**
			2	6369.5	1076.0	*4442.4*	6345.1	5511.1	6458.8	4812.9 ± 142.2	**4168.0 ± 30.9**
		0.6	0	6528.2	510.2	*2245.0*	3602.5	2790.3	3496.1	3160.1 ± 125.9	**2240.2 ± 10.2**
			1	8161.3	613.5	*3262.9*	3430.4	3838.4	3890.5	3259.4 ± 104.6	**2746.1 ± 55.7**
			2	2916.5	435.1	*1849.5*	2507.8	2270.4	2891.9	2039.9 ± 32.9	**1803.8 ± 18.4**
	7	0.3	0	8078.1	1817.0	*3455.5*	3516.0	3499.2	4121.9	3325.4 ± 42.2	**3323.2 ± 60.3**
			1	9783.2	2744.2	4630.2	**4374.2**	4977.1	4754.0	4647.3 ± 165.9	4424.8 ± 44.5
			2	6068.6	1771.0	*2760.9*	2978.4	3017.9	2969.4	2788.8 ± 43.1	**2704.9 ± 32.6**
		0.6	0	-	581†	*3352.9*	4658.9	3774.8	4642.0	4697.4 ± 70.1	**3351.3 ± 3.8**
			1	-	374†	*1836.3*	2537.1	2146.1	2844.5	2182.6 ± 74.0	**1819.8 ± 3.9**
			2	-	508†	**2377.1**	3618.7	3159.9	3900.3	3644.1 ± 109.9	**2377.1 ± 0.0**

CPLEX columns also details if: optimal solution found (*); only feasible solution found is LP relaxation of root node (†). Each row highlights the best solution (bold) and the best heuristic solution (italic)

Appendix B: Exact method comparison

Table 9 Comparison of exact method solvers for instances with 10 jobs and 7 machines. Run times are limited to one hour

Workers	Flex	Rep	CPLEX		Gurobi			
					Complete		Heuristic	
			Upper bound	Lower bound	Upper bound	Lower bound	Upper bound	Lower bound
2	0.3	0	**90.7**	85.3	**90.7**	82.8	**90.7**	81.8
		1	189.5	109.9	**187.2**	109.7	192.4	107.1

Table 9 continued

Workers	Flex	Rep	CPLEX		Gurobi			
					Complete		Heuristic	
			Upper bound	Lower bound	Upper bound	Lower bound	Upper bound	Lower bound
		2	54.8	43.0	**54.4**	48.0	**54.4**	49.0
	0.6	0	125.6	90.5	123.2	80.9	**119.8**	79.7
		1	111.3	80.5	112.2	78.6	**104.4**	76.0
		2	129.3	125.5	128.4	125.5	**128.1**	125.5
3	0.3	0	**108.1**	108.1	**108.1**	108.1	**108.1**	108.1
		1	126.9	90.9	123	100.3	**122.3**	94.8
		2	**96**	91.3	96.4	82.9	96.4	82.5
	0.6	0	**59.9**	55.2	**59.9**	55.0	**59.9**	55
		1	90.6	66.0	87.4	66.1	**87.1**	62.3
		2	**71.2**	62.6	71.7	60.7	73.8	58.6
7	0.3	0	**102.5**	102.5	**102.5**	102.5	**102.5**	102.3
		1	**119.5**	117.0	**119.5**	119.5	**119.5**	118
		2	**93**	82.0	**93**	84.0	**93**	79.4
	0.6	0	99.3	96.1	99.1	96.2	**99.1**	96.1
		1	97.1	80.0	96.1	83.0	**95.4**	83
		2	**82.2**	79.0	**82.2**	61.3	**82.2**	61.1

References

Ahmadi, E., Zandieh, M., Farrokh, M., Emami, S.M.: A multi objective optimization approach for flexible job shop scheduling problem under random machine breakdown by evolutionary algorithms. Comput. Oper. Res. **73**, 56–66 (2016)

Akbar, M., Irohara, T.: Metaheuristics for the multi-task simultaneous supervision dual resource-constrained scheduling problem. Eng. Appl. Artif. Intell. **96**(March), 104004 (2020)

Bitar, A., Dauzère-Pérès, S., Yugma, C., Roussel, R.: A memetic algorithm to solve an unrelated parallel machine scheduling problem with auxiliary resources in semiconductor manufacturing. J. Sched. **19**(4), 367–376 (2016)

Burdett, R.L., Corry, P., Eustace, C., Smith, S.: A flexible job shop scheduling approach with operators for coal export terminals - A mature approach. Comput. Oper. Res. **115**, 104834 (2020)

Burdett, R.L., Corry, P., Yarlagadda, P.K., Eustace, C., Smith, S.: A flexible job shop scheduling approach with operators for coal export terminals. Comput. Oper. Res. **104**, 15–36 (2019)

Cakici, E., Mason, S.J.: Parallel machine scheduling subject to auxiliary resource constraints. Product. Plan. Control **18**(3), 217–225 (2007)

Chaudhry, I.A., Khan, A.A.: A research survey: Review of flexible job shop scheduling techniques. Int. Trans. Oper. Res. **23**(3), 551–591 (2016)

Ciro, G.C., Dugardin, F., Yalaoui, F., Kelly, R.: A fuzzy ant colony optimization to solve an open shop scheduling problem with multi-skills resource constraints. *IFAC-PapersOnLine (Special Issue: 15th IFAC Symposium onInformation Control Problems inManufacturing: INCOM 2015)*, 28(3):715–720 (2015)

Coito, T., Martins, M.S., Viegas, J.L., Firme, B., Figueiredo, J., Vieira, S.M., Sousa, J.M.: A middleware platform for intelligent automation: An industrial prototype implementation. Comput. Ind. **123**, 103329 (2020)

Costigliola, A., Ataíde, F.A., Vieira, S.M., Sousa, J.M.: Simulation Model of a Quality Control Laboratory in Pharmaceutical Industry. IFAC-PapersOnLine **50**(1), 9014–9019 (2017)
CResults SMART-QC: QC Labs Resource Planning, Scheduling and Cost of Quality (COQ) (2019)
Cunha, M. M., Viegas, J. L., Martins, M. S. E., Coito, T., Costigliola, A., Figueiredo, J., Sousa, J. M. C., Vieira, S. M.: Dual Resource Constrained Scheduling for Quality Control Laboratories. *Proceedings of the 9th IFAC/IFIP/IFORS/IISE/INFORMS Conference on Manufacturing Modelling, Management and Control (MIM 2019)* (2019)
Dhiflaoui, M., Nouri, H.E., Driss, O.B.: Dual-resource constraints in classical and flexible job shop problems: a state-of-the-art review. Procedia Comput. Sci. **126**, 1507–1515 (2018)
El Maraghy, H., Patel, V., Abdallah, I.B.: Scheduling of manufacturing systems under dual-resource constraints using genetic algorithms. J. Manuf. Syst. **19**(3), 186–201 (2000)
Flores-Luyo, L., Agra, A., Figueiredo, R., Ocaña, E.: Heuristics for a vehicle routing problem with information collection in wireless networks. J. Heuristics **26**(2), 187–217 (2020)
Garey, M.R., Johnson, D.S., Sethi, R.: The complexity of Flowshop and jobshop scheduling. Math. Oper. Res. **1**(2), 117–129 (1976)
Geigert, J.: Quality Assurance and Quality Control for Biopharmaceutical Products. In *Development and Manufacture of Protein Pharmaceuticals*, pages 361–404. Springer (2002)
Guimarães, K.F., Fernandes, M.A.: An approach for flexible Job-Shop Scheduling with separable sequence-dependent setup time. Conf. Proc. - IEEE Int. Conf. Syst, Man Cyber. **5**, 3727–3731 (2007)
Ho, N.B., Tay, J.C., Lai, E.M.: An effective architecture for learning and evolving flexible job-shop schedules. Eur. J. Oper. Res. **179**(2), 316–333 (2007)
Kress, D., Müller, D., Nossack, J.: A worker constrained flexible job shop scheduling problem with sequence-dependent setup times. OR Spectrum **41**(1), 179–217 (2019)
Lei, D., Guo, X.: Variable neighbourhood search for dual-resource constrained flexible job shop scheduling. Int. J. Prod. Res. **52**(9), 2519–2529 (2014)
Lopes, M.R., Costigliola, A., Pinto, R., Vieira, S., Sousa, J.M.: Pharmaceutical quality control laboratory digital twin-A novel governance model for resource planning and scheduling. Int. J. Prod. Res. **58**(21), 6553–6567 (2020)
Lopes, M.R., Costigliola, A., Pinto, R.M., Vieira, S.M., Sousa, J.M.: Novel governance model for planning in pharmaceutical quality control laboratories. IFAC-PapersOnLine **51**(11), 484–489 (2018)
Lv, Y., Lin, D.: Design an intelligent real-time operation planning system in distributed manufacturing network. Indus. Manag. Data Syst. **117**(4), 742–753 (2017)
Malve, S., Uzsoy, R.: A genetic algorithm for minimizing maximum lateness on parallel identical batch processing machines with dynamic job arrivals and incompatible job families. Comput. Oper. Res. **34**(10), 3016–3028 (2007)
Martins, M.S., Viegas, J.L., Coito, T., Firme, B.M., Sousa, J.M., Figueiredo, J., Vieira, S.M.: Reinforcement learning for dual-resource constrained scheduling. IFAC-PapersOnLine **53**(2), 10810–10815 (2020)
Maslaton, R.: Resource scheduling in QC laboratories. Pharm. Eng. **32**(6), 68–73 (2012)
Mati, Y., Lahlou, C., Dauzère-Pérès, S.: Modelling and solving a practical flexible job-shop scheduling problem with blocking constraints. Int. J. Prod. Res. **49**(8), 2169–2182 (2011)
Montgomery, D. C.: *Introduction to Statistical Quality Control*. Wiley, fifth edition edition (2005)
Paksi, A. B., Ma'Ruf, A.: Flexible Job-Shop Scheduling with Dual-Resource Constraints to Minimize Tardiness Using Genetic Algorithm. *IOP Conference Series: Materials Science and Engineering*, 114(1) (2016)
Panwalkar, S.S., Iskander, W.: A survey of scheduling rules. Oper. Res. **25**(1), 45–61 (1977)
Pinedo, M. L.: *Scheduling - Theory, Algorithms, and Systems*. Springer, fifth edition edition (2016)
Roshanaei, V., Azab, A., El Maraghy, H.: Mathematical modelling and a meta-heuristic for flexible job shop scheduling. Int. J. Prod. Res. **51**(20), 6247–6274 (2013)
Ruiz-Torres, A.J., Ablanedo-Rosas, J.H., Otero, L.D.: Scheduling with multiple tasks per job - The case of quality control laboratories in the pharmaceutical industry. Int. J. Prod. Res. **50**(3), 691–705 (2012)
Ruiz-Torres, A.J., Mahmoodi, F., Kuula, M.: Quality assurance laboratory planning system to maximize worker preference subject to certification and preference balance constraints. Comput. Oper. Res. **83**, 140–149 (2017)
Saidi-Mehrabad, M., Fattahi, P.: Flexible job shop scheduling with tabu search algorithms. Int. J. Adv. Manuf. Technol. **32**(5–6), 563–570 (2007)
Schäfer, R.: Concepts for dynamic scheduling in the laboratory. J. Lab. Autom. **9**(6), 382–397 (2004)

Scholz-reiter, B., Heger, J., Hildebrandt, T.: Analysis And Comparison Of Dispatching Rule- Based Scheduling In Dual-Resource Constrained Shop-Floor Scenarios. World Congr. Eng. Comput. Sci. **2**(October), 1–7 (2009)

Shen, L., Dauzère-Pérès, S., Neufeld, J.S.: Solving the flexible job shop scheduling problem with sequence-dependent setup times. Eur. J. Oper. Res. **265**(2), 503–516 (2018)

Zhang, H., Cai, S., Luo, C., Yin, M.: An efficient local search algorithm for the winner determination problem. J. Heuristics **23**(5), 367–396 (2017)

Zheng, M., Wu, K.: Smart spare parts management systems in semiconductor manufacturing. Indus. Manag. Data Syst. **117**(4), 754–763 (2017)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.